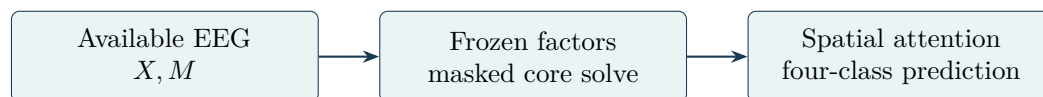


COLLABORATOR ORIENTATION

Tensor representations for EEG channel-loss robustness

AGFL-inm: concept, mathematics, and study design



Central question. Can a shared, low-rank channel–feature model preserve classification information when known EEG electrodes disappear, compared with attention receiving the same available features directly?

Working hypothesis. Training-fitted structure may reduce degradation under electrode loss. It may also discard useful information or hurt full-input accuracy. The experiment is designed to measure both possibilities.

This is an explanation of the supplied code and configuration, not an experimental-results paper. No accuracy gain has been measured or verified in this documentation task. The original proposal and current implementation are explicitly distinguished.

Reviewed source: 84ae174 (v0.13)

Review date: 1 October 2026

Primary entry points: `run.py`, `configs/study.json`

Editable source: `docs/concept/agfl_concept.tex`

1 Research question and scope

EEG records voltage signals at named scalp electrodes. The supplied study classifies four motor-imagery tasks: left hand, right hand, both feet, and tongue. BCI Competition IV 2a provides nine participants, 22 EEG channels, and recordings at 250 Hz; three EOG channels are excluded from classifier inputs [1].

An electrode can be unavailable throughout a trial or disappear and return between windows. The classifier receives the availability mask explicitly. The question concerns robustness to known missing sensors, rather than discovery of unknown sensor identities or recovery of raw waveforms.

1.1 Why use a tensor?

For one trial, collect window-local learned features in

$$X \in \mathbb{R}^{C \times P \times F}, \quad M \in \{0, 1\}^{C \times P}, \quad (C, P, F) = (22, 4, 32). \quad (1)$$

Here $M_{cp} = 1$ means electrode c is observed in window p . Electrode positions remain fixed when channels disappear. The mask broadcasts over the feature index f ; absence is distinct from a measured numerical zero.

Shared Tucker factors describe channel and feature structure. A new trial’s core is inferred from only its observed entries. Attention then receives a fixed set of latent spatial tokens. Tensor decomposition provides a structured model of the feature array [2]; arranging features in an array alone does not provide a reconstruction mechanism.

1.2 What is implemented?

- An EEGNet-derived, channel/window-local encoder shared and frozen across all downstream arms [3].
- Baseline and Tucker-core representations with MHA and Performer [4, 5].
- MHA controls for tensor completion, separable linear compression, and training-mean completion.
- Full and mixed downstream training; evaluation with static and dynamic random or spatially grouped channel loss.

The current study is participant-specific and within-session. It does not implement the proposal’s subject-disjoint folds, fixed spectral features, temporal-position tokens, or Nyström attention. Signal Transformer is present in the library but is outside this study’s comparison matrix.

Reader’s route. The following pages explain preprocessing and calibration, the factor model, exact inference, attention, outage masks, the experiment matrix, evaluation, and the limits of the implementation.

Implementation: `docs/project_overview.md`; `configs/study.json`; `references/proposal_extracted.txt`

2 Pipeline and separation of fitting from inference

Each usable trial starts at the cue and contains 1,000 samples per electrode. Four nonoverlapping 250-sample windows each represent one second. The loader excludes artifact-marked, boundary-crossing, and unusable/nonfinite trials.

Each window is filtered independently using a fourth-order 2–30 Hz Butterworth SOS zero-phase filter. This is offline processing. Independent windows prevent filtering across a hidden interval; the protocol is not causal online inference.

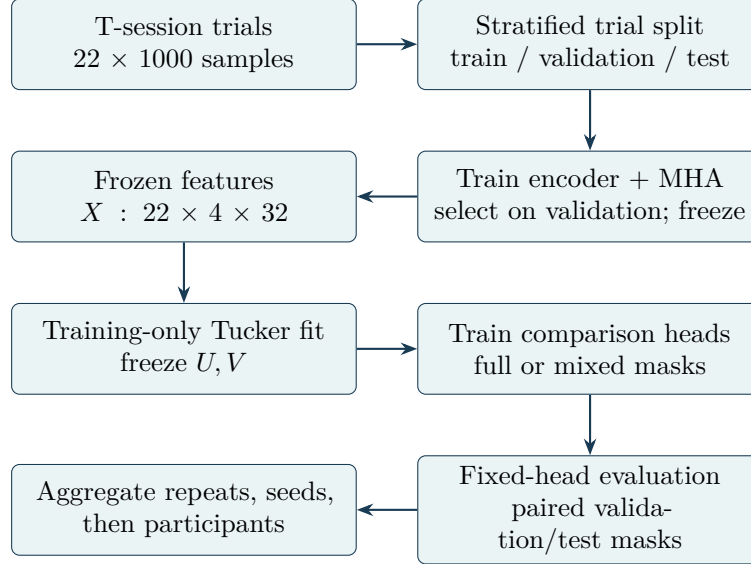


Figure 1: Data flow. Normalization and tensor fitting use training trials only. Validation selects supervised epochs; test data only score the fixed models.

2.1 Shared feature source

The encoder applies shared temporal convolution (16 filters, kernel 32), a grouped 1×1 expansion with depth multiplier 2, and separable temporal convolution with 32 output filters. Pooling by 8 and 16 leaves one temporal sample per window, yielding 32 features. It does not convolve across electrodes or window boundaries and is an adaptation of EEGNet, not standard EEGNet’s full-head spatial convolution.

For each participant/seed, full-input MHA-supervised pretraining selects the encoder on full-input validation. The pretraining head is discarded; the encoder is frozen, including BatchNorm statistics and dropout behavior. Raw channel normalization and subsequent channel-feature standardization use training statistics only. All arms share the resulting feature cache.

The cache contains full trial features for synthetic outages. This is consistent with the declared simulation because frozen encoding is local to each channel/window and hidden features are removed before head or core inference. It does not simulate permanently absent channels during encoder calibration.

Implementation: `inm/data.py`; `inm/model.py::EEGWindowEncoder`; `inm/study.py::_calibrate`

3 The shared Tucker-2 model

Compress the channel and feature modes while leaving the window axis explicit:

$$U \in \mathbb{R}^{C \times R_C}, \quad V \in \mathbb{R}^{F \times R_F}, \quad G \in \mathbb{R}^{R_C \times P \times R_F}, \quad (2)$$

$$\hat{X}_{cpf} = \sum_{r=1}^{R_C} \sum_{s=1}^{R_F} U_{cr} G_{rps} V_{fs}, \quad \hat{X}_p = U G_p V^\top. \quad (3)$$

The configuration declares $R_C = R_F = 4$. Factors U, V are shared across trials; the core G is specific to each trial and its mask. No factor is learned along time, and no core solve smooths through future windows.

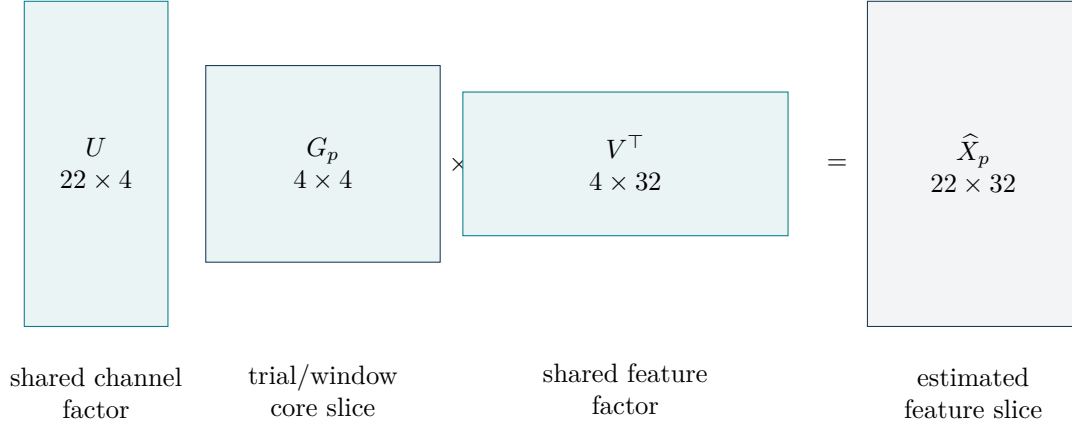


Figure 2: One window of Tucker-2 reconstruction. Four separate core slices retain the four windows. Boxes are schematic; dimensions are labeled explicitly.

3.1 Compression and its meaning

The original feature array has $22 \cdot 4 \cdot 32 = 2,816$ scalar entries per trial. Its core has $4 \cdot 4 \cdot 4 = 64$ entries, a 44-fold reduction in this representation count. The two shared factors contain $22 \cdot 4 + 32 \cdot 4 = 216$ scalars, plus fitted-state bookkeeping.

At attention input, the baseline has 22 tokens per window with 32 source features each; the core has four tokens with four source features each. Both token adapters project to attention dimension 32. This reduction does not measure end-to-end memory, trainable parameter savings, or runtime: core inference, the feature cache, token adapters, and attention implementation also contribute.

3.2 Factor coordinates are not anatomy

Columns are projected to unit Euclidean norm, but are not required to be orthogonal. Components are learned latent coordinates, not named brain regions or anatomical connectivity. Factor/core parameterizations admit ambiguities; freezing the shared factors keeps the coordinates consistent across trial inference.

Low-rank reconstruction may suppress noise, but it may also suppress class information. A fixed latent shape and a low reconstruction error are insufficient evidence of improved classification.

Implementation: `inm/tensor_attention.py::Tucker2`; `configs/study.json::tensor`

4 Exact core inference from observed features

For window p , let $O_p = \{c : M_{cp} = 1\}$, $n_p = |O_p|$, and $D_p = n_p F$, the number of observed scalar features. Holding factors fixed, infer the core by normalized ridge regression:

$$G_p^* = \arg \min_{G \in \mathbb{R}^{R_C \times R_F}} \ell_p(G), \quad \ell_p(G) = \frac{\|X_{O_p, p, :} - U_{O_p, :} G V^\top\|_F^2}{D_p} + \lambda \|G\|_F^2. \quad (4)$$

The declared ridge is $\lambda = 10^{-3}$. Hidden channels do not occur in the data term; inference uses neither labels nor hidden reference features.

4.1 Normal equations

Differentiating Equation (4) and multiplying by $D_p/2$ gives

$$A_p G_p B + D_p \lambda G_p = T_p, \quad A_p = U_{O_p, :}^\top U_{O_p, :}, \quad B = V^\top V, \quad T_p = U_{O_p, :}^\top X_{O_p, p, :} V. \quad (5)$$

The observation normalization matters: the ridge coefficient in the normal equation is $D_p \lambda$, rather than λ alone.

Let $A_p = Q_p \text{diag}(a_p) Q_p^\top$ and $B = S \text{diag}(b) S^\top$ be symmetric eigendecompositions. With $H_p = Q_p^\top T_p S$, Equation (5) reduces to entrywise division:

$$(\tilde{G}_p)_{ij} = \frac{(H_p)_{ij}}{(a_p)_i b_j + D_p \lambda}, \quad G_p^* = Q_p \tilde{G}_p S^\top. \quad (6)$$

Gram matrices are symmetrized, roundoff-negative eigenvalues clipped to zero, and solves computed in float64. With $n_p \geq 1$ and positive ridge, every denominator is positive, so the conditional core is unique even with deficient observed-channel rank. Statistical recovery is a separate question.

4.2 Numerical and missingness safeguards

Select observed entries before any arithmetic:

$$X_{cpf}^{\text{safe}} = \begin{cases} X_{cpf}, & M_{cp} = 1, \\ 0, & M_{cp} = 0. \end{cases}$$

The implementation uses `torch.where`. Multiplication is unsuitable for unknown placeholders because $0 \cdot \text{NaN} = \text{NaN}$. Observed features must be finite and every trial/window must retain a channel. The low-level solver accepts any positive observed count; the declared study uses at least six. Core inference is detached and does not update the factors.

4.3 Learning the factors

On training trials only, minimize

$$\frac{1}{N_{\text{train}} P} \sum_{n \in D_{\text{train}}} \sum_{p=1}^P \ell_p(G_{np}; U, V, X_n, M_n), \quad \|U_{:r}\|_2 = \|V_{:s}\|_2 = 1.$$

Each minibatch alternates exact core solves and Adam factor updates with cores held fixed, followed by column normalization. The study uses 30 epochs on full training features. This nonconvex projected alternating fit has no stated global optimality guarantee. Fitted factors become buffers outside classifier optimizers.

Implementation: `inm/tensor_attention.py::_solve, _objective, fit, encode`

5 Representations, spatial attention, and readout

The tensor route uses G directly. Its separate completion control fills missing features while preserving observed values exactly:

$$X_{cpf}^{\text{comp}} = \begin{cases} X_{cpf}, & M_{cp} = 1, \\ (UG_p V^\top)_{cf}, & M_{cp} = 0. \end{cases} \quad (7)$$

Completion estimates features, not raw EEG. Its original mask remains an input. The linear control computes $G_p^{\text{lin}} = W_C X_p^{\text{safe}} W_F^\top$, with trainable separable maps and the same core shape. Mean completion inserts the training channel-feature mean, which is close to zero after standardization.

5.1 Tokens and availability information

For window p , let t_{jp} denote the represented feature vector and $m_p = M_{\cdot p}$. Tokens are constructed as

$$z_{jp} = W_{\text{in}} t_{jp} + b_{\text{in}} + a_j + w_{\text{status}} s_{jp} + W_M m_p. \quad (8)$$

Here a_j is a learned electrode or component identity. For electrode tokens, $s_{jp} = M_{jp}$; for latent tokens, $s_{jp} = n_p/C$. Baseline missing tokens replace the projected feature term with a learned placeholder. All arms receive the full observed-channel mask; there is no temporal position term.

5.2 Two attention families

Each window is processed independently by a residual attention/FFN block. For one MHA head, with $d_h = 32/4 = 8$, standard attention is [4]

$$Q = ZW_Q, \quad K = ZW_K, \quad L = ZW_L, \quad A(Z) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right) L. \quad (9)$$

The value symbol L avoids confusing attention values with tensor factor V . Performer approximates the softmax kernel through a positive random-feature map ϕ , allowing the reassociation [5]

$$A_i(Z) \approx \frac{\phi(q_i)^\top (\phi(K)^\top L)}{\phi(q_i)^\top \sum_j \phi(k_j)}. \quad (10)$$

The implementation uses 64 fixed orthogonal random features per head, stabilized exponentials, and a denominator floor. Neither attention module receives an exact key-padding mask; missingness enters through token construction.

5.3 Pooling and prediction

Let h_{jp} be the final normalized tokens and w_{jp} the pooling weights:

$$h_p = \frac{\sum_j w_{jp} h_{jp}}{\sum_j w_{jp}}, \quad \bar{h} = \frac{1}{P} \sum_p h_p, \quad \pi = \text{softmax}(W_o \bar{h} + b_o). \quad (11)$$

Baseline pooling counts observed electrodes only, though missing placeholders can still participate in attention. Completion pools all electrode tokens; core routes pool all latent tokens. No declared window is completely absent. Training uses cross-entropy and AdamW. Window averaging implies invariance to a joint permutation of feature and mask windows at evaluation.

Implementation: `inm/model.py::FeatureClassifier`; `agfl/attention/mha/layer.py`;
`agfl/attention/performer/layer.py`

6 Availability protocol: static and dynamic loss

Retain $k \in \{22, 16, 11, 6\}$ electrodes per window. Missing percentages are $100(22 - k)/22$, giving 0%, 27.27%, 50%, and 72.73%. Six electrodes are the supplied integer approximation to roughly three-quarters channel loss.

Pattern	How missing channels are chosen
Random static	A random subset is kept for every window of a trial.
Spatial static	The nearest $22 - k$ electrodes to a sampled center are removed on fixed schematic scalp coordinates.
Dynamic random	Distinct random subsets follow an A–B–A schedule.
Dynamic spatial	Distinct spatial-loss subsets follow the same schedule.

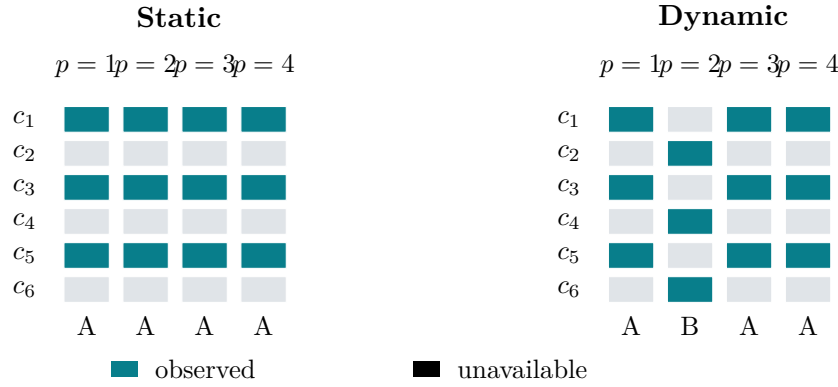


Figure 3: Illustrative six-channel universe retaining three channels, rather than an actual BCI mask. The same observed count is kept in every window. For the study’s four windows, the dynamic schedule is A, B, A, A.

The general dynamic rule uses B on $[\lfloor P/3 \rfloor, \lfloor 2P/3 \rfloor]$ in zero-based window indices and A elsewhere. Reappearing channels retain their original identities. Schematic spatial coordinates define clustered loss, not measured anatomy.

6.1 Training and evaluation masks

Full training observes all 22 channels. Mixed training draws 22, 16, or 6 uniformly per trial/epoch; degraded trials choose random static or random dynamic loss uniformly. Eleven-channel inputs and spatial patterns are evaluation-only.

SHA256-based keys use participant, seed, partition, trial identity, condition, and repeat or training epoch, excluding attention and representation identity. Nonfull channel subsets are canonically assigned to train, validation, or test by hashing their original electrode bitset modulo three. Rejection sampling enforces that assignment across patterns; the full subset is the shared exception. Thus compared arms receive identical masks without reusing nonfull subsets across data partitions.

Implementation: `inm/availability.py::make_mask_bank, training_mask_bank, subset_partition`

7 The declared experiment

For each participant A01–A09 and seed 0, 1, or 2, stratify that participant’s T-session trials into approximately 60% training, 20% validation, and 20% test. Fractions are rounded within class. No participants are pooled during training.

Representation	MHA	Performer	Regimes
Baseline	yes	yes	full, mixed
Tucker core	yes	yes	full, mixed
Tensor completion	yes	—	full, mixed
Separable linear core	yes	—	full, mixed
Training-mean completion	yes	—	full, mixed

There are $(2 \cdot 2 + 3) \cdot 2 = 14$ heads per participant/seed and $9 \cdot 3 = 27$ tasks, totaling 378 downstream classifier fits. In addition, there are 27 shared encoder fits and 27 Tucker calibrations. Each calibrated encoder/factor pair is reused by all 14 downstream heads in its task.

7.1 Budgets and selection

Setting	Supplied value
Supervised training	Cross-entropy, AdamW, batch size 32, at most 250 epochs
Learning rate	Target 10^{-3} ; 10-epoch warmup, then cosine schedule
Regularization	Weight decay 10^{-4} , gradient clipping norm 1
Early stopping	At least 75 epochs; patience 50
Encoder dropout	0.5
Head	Dimension 32, four heads, one attention/FFN block, dropout 0.1
Factor fitting	30 epochs; learning rate 0.01; batch size 64; ridge 0.001
Mask repeats	One full-input condition; five repeats per degraded condition

The best full-input validation balanced accuracy selects the epoch; unweighted validation log loss breaks ties. Both downstream availability regimes use full-input validation for selection. There is no test-based rank search or arm-specific tuning in the supplied configuration.

7.2 Evaluation coverage and controls

One full condition and four patterns at each of three degraded counts give 13 scenarios. Repeats produce $1 + 3 \cdot 4 \cdot 5 = 61$ evaluations per partition per head, or 122 across validation and test. Heads are fixed; individual outage conditions do not trigger retraining.

Completion tests imputation in electrode space. Linear-core control tests supervised separable compression. Mean completion provides simple imputation. These controls differ in objectives, capacity, token count, and pooling; equal latent shape does not imply equal trainable parameter counts.

Implementation: `configs/study.json`; `inm/protocol.py::arms, tasks`; `inm/training.py::fit`

8 Outcomes and paired statistical comparisons

For four classes, balanced accuracy averages class recalls:

$$\text{BA} = \frac{1}{4} \sum_{a=1}^4 \frac{TP_a}{TP_a + FN_a}. \quad (12)$$

Macro-F1 averages per-class F1, $F1_{\text{macro}} = \frac{1}{4} \sum_a 2TP_a / (2TP_a + FP_a + FN_a)$, with metric edge cases handled by the implementation. Balanced accuracy is primary; accuracy, macro-F1, and reconstruction are secondary.

8.1 Aggregation and gains

Let $b_{sekr}^{(m)}$ be balanced accuracy for method m , participant s , seed e , retained count k , mask repeat r , and loss pattern g , for a fixed attention family and training regime. First average mask repeats, then three seeds, then nine participant means:

$$\bar{b}_{skg}^{(m)} = \frac{1}{3} \sum_{e=1}^3 \frac{1}{R_{kg}} \sum_{r=1}^{R_{kg}} b_{sekr}^{(m)}, \quad S_{kg}^{(m)} = \frac{1}{9} \sum_{s=1}^9 \bar{b}_{skg}^{(m)}. \quad (13)$$

Participants have equal weight. Neither masks nor seeds are independent participants. For matched Tucker-core and baseline heads, define percentage-point gains

$$\Delta_{skg} = 100(\bar{b}_{skg}^{(T)} - \bar{b}_{skg}^{(B)}), \quad \Delta_{kg} = \frac{1}{9} \sum_s \Delta_{skg}. \quad (14)$$

Positive gain favors the tensor route. Positive degradation means a drop from the method's full-input score: $D_{kg}^{(m)} = 100(S_{22}^{(m)} - S_{kg}^{(m)})$. For each loss pattern, the robustness summary is

$$\mathcal{R}_g^{(m)} = \frac{1}{4} (S_{22}^{(m)} + S_{16,g}^{(m)} + S_{11,g}^{(m)} + S_{6,g}^{(m)}). \quad (15)$$

It is an equal-weight four-condition average, not a continuous-curve area.

8.2 Uncertainty and incomplete results

Resample the nine participant-level paired differences with replacement 2,000 times. The 2.5th and 97.5th percentiles of resampled means form pointwise 95% bootstrap intervals. Subject standard deviation is reported separately. These intervals are exploratory and not multiple-comparison corrected. Paired mask hashes must agree; incompatible pairs are excluded visibly. All-participant means remain blank until every required fit is complete.

8.3 Hidden-feature reconstruction

Only after inference, evaluate artificial hidden entries against reference features:

$$\text{NRMSE}_{\text{hid}} = \sqrt{\frac{\sum_{c,p,f:M_{cp}=0} (\hat{X}_{cpf} - X_{cpf})^2}{\sum_{c,p,f:M_{cp}=0} X_{cpf}^2}}. \quad (16)$$

The study pools sums over the evaluated partition in standardized feature space. No hidden entries or zero reference energy yields an undefined/blank value. This exact implementation has no additive epsilon in the denominator. Reconstruction scores neither fit factors nor select heads.

Implementation: `inm/training.py::metrics`; `inm/reporting.py`; `inm/study.py::_reconstruction_scores`

9 Proposal fidelity, limitations, and collaboration priorities

Original proposal	Supplied implementation
Fixed channel-wise time–frequency features	Frozen supervised EEGNet-derived window features
Three subject-disjoint folds	Nine individually trained participants; within-T-session trial splits
Component–window tokens with time positions	Spatial attention separately per window; order-invariant averaging
MHA, Nyström-style, Performer-style	MHA and Performer; no Nyström implementation
Small validation-only rank search	Fixed ranks $R_C = R_F = 4$, without an executed rank search
Tensor core and completion controls	Both, plus separable linear and training-mean controls
Varying availability and held-out patterns	Full/mixed head training; held-out 11-channel and spatial losses

9.1 What the study can establish

If executed correctly, the study can compare frozen-representation behavior under declared channel losses and quantify full-input costs, per-participant variation, and paired core-versus-baseline changes. It can examine whether compression, completion, or mixed-mask training explains robustness.

It cannot establish unseen-participant generalization, the official train-on-T / test-on-E benchmark, causal online performance, anatomical connectivity, or a gain for standard end-to-end EEGNet. Full-channel calibration is assumed. MHA-based feature pretraining may favor MHA. Severe spatial loss can remove information that no factor model can reconstruct. The short spatial sequences do not themselves support a Performer speed claim.

9.2 Readiness at the reviewed workstation

The dependency-light task planner runs and confirms 27 tasks and 378 heads. Python 3.13.13 and the LaTeX toolchain are available. PyTorch, NumPy, SciPy, scikit-learn, and MNE are absent; the default `../ml` data directory and saved results are absent. Consequently, numerical inference, training, CUDA behavior, and performance are unverified here. No checked-in tests or Slurm launcher were found. Dataset helpers are present; a broad ignore rule hides them from ordinary file searches, so use the tracked Git inventory.

9.3 Suggested next steps with the supervisor

1. Agree on the intended scientific protocol: this participant-specific study, the original subject-disjoint proposal, or a separate cross-session design.
2. Provision the scientific environment and official GDF recordings on the experiment machine; inspect a single participant/seed task first.
3. Add numerical checks for the ridge solve, hidden-NaN isolation, observed completion values, mask separation, and incomplete-pair reporting.
4. Preserve source/configuration/data identities, report unfavorable outcomes, and interpret controls before attributing improvements to tensor structure.

Implementation: `references/proposal_extracted.txt`; `docs/investigation.md`; `docs/experiment.md`

10 Reproducing the document and navigating the repository

Orientation files. `docs/README.md` indexes the existing technical notes. `docs/onboarding.md` provides a guided collaborator workflow; `docs/investigation.md` records dated evidence and readiness. Root `AGENTS.md` supplies future assistant context and research invariants.

Code ownership. `inm/` owns the experiment, masking, representations, training orchestration, reporting, and plots. `agfl/` owns EEG model, attention, data/split, optimization, and reproducibility helpers. `references/` contains the proposal as PDF, DOCX, and extracted text.

Useful commands from the repository root:

```
# Dependency-light plan; no training
python run.py --config configs/study.json --plan

# On a provisioned CUDA/data machine: A01, seed 0, all 14 heads
python run.py --config configs/study.json --task-index 0

# After saved results exist
python run.py --config configs/study.json --summarize-only
python run.py --config configs/study.json --summarize-only --plots

# Rebuild this PDF without Python or external images
make -C docs/concept
```

The report entry point is `results/inm-v2/report/summary.md`. Keep `artifacts/` for calibration, splits, histories, and provenance. Selected head weights are evaluated in memory and are not saved as standalone checkpoints. Changed code/packages/configuration require a new compatible study output directory; do not mix incompatible runs.

References

- [1] C. Brunner et al. *BCI Competition 2008: Graz data set A (BCI Competition IV, data set 2a)*. Official dataset description. bbci.de/competition/iv/desc_2a.pdf.
- [2] T. G. Kolda and B. W. Bader. *Tensor Decompositions and Applications*. SIAM Review 51(3), 455–500, 2009. [Author’s publication page](#); doi:10.1137/07070111X.
- [3] V. J. Lawhern et al. *EEGNet: A Compact Convolutional Network for EEG-based Brain–Computer Interfaces*. Journal of Neural Engineering, 2018. [arXiv:1611.08024](#).
- [4] A. Vaswani et al. *Attention Is All You Need*. NeurIPS, 2017. [arXiv:1706.03762](#).
- [5] K. Choromanski et al. *Rethinking Attention with Performers*. ICLR, 2021. [arXiv:2009.14794](#).
- [6] *Tensor-Structured Attention for Classification under Variable Signal Availability*. Repository proposal in `references/`; author/date unspecified in the extracted text.
- [7] AGFL-inm repository, reviewed revision 84ae174, `configs/study.json`, `inm/`, `agfl/`, and `docs/`. Reviewed 1 October 2026. Equations and settings describe this source snapshot.